

Encerramento de Auditoria — Condominio-7663

Commit final: 7471e3d | Data: 2026-06-11 | Classificação: Produção ☐

Declaração de Encerramento

A aplicação está APROVADA para produção. Todos os blockers foram resolvidos. Os mecanismos de fallback estão activos e validados. Não existem crashes, loops infinitos, ou dados incorrectos no fluxo principal.

Auditoria concluída sobre o commit 7471e3d . Quatro ciclos de fix iterativos (2bf5800 → 77258a7 → b509c91 → 0df40a2 → 7471e3d) resolveram todos os problemas identificados.

Fluxograma — Ciclo de Vida dos Dados (Sync → Render)

TRIGGER: Admin autentica no browser



ProtectedRoute.tsx – useEffect

Guarda 1: syncedRef.current === false ? —NO→ SKIP

Guarda 2: sessionStorage("bank-sync-done") ausente? —NO→ SKIP

Ambas OK → syncedRef.current = true

→ sessionStorage.setItem("bank-sync-done", "1")

→ silentBankSync(queryClient) [fire & forget]

▼ [POST /api/bank/sync]

bank.ts – /api/bank/sync

1. Enable Banking API → busca transacções (dateFrom..dateTo)

2. importTransactions(allTransactions)

└ Classifica cada tx (quota / despesa / portão / etc.)

└ Upsert em schema.quotas / schema.despesas

└ Retorna { quotasCreated, despesasCreated, errors }

3. recalcularSaldos() ← chamada explícita após import

└ Lê quotas/despesas da DB

└ Calcula saldos derivados (obras, portão, incêndio, extra)

└ Persiste em schema.configuracoes (upsert por chave)

4. db.insert(bankSyncLogs) – registo do sync

| resposta 200



silentBankSync (client) – após await fetch(...)

await new Promise(r => setTimeout(r, 500)) // espera write DB

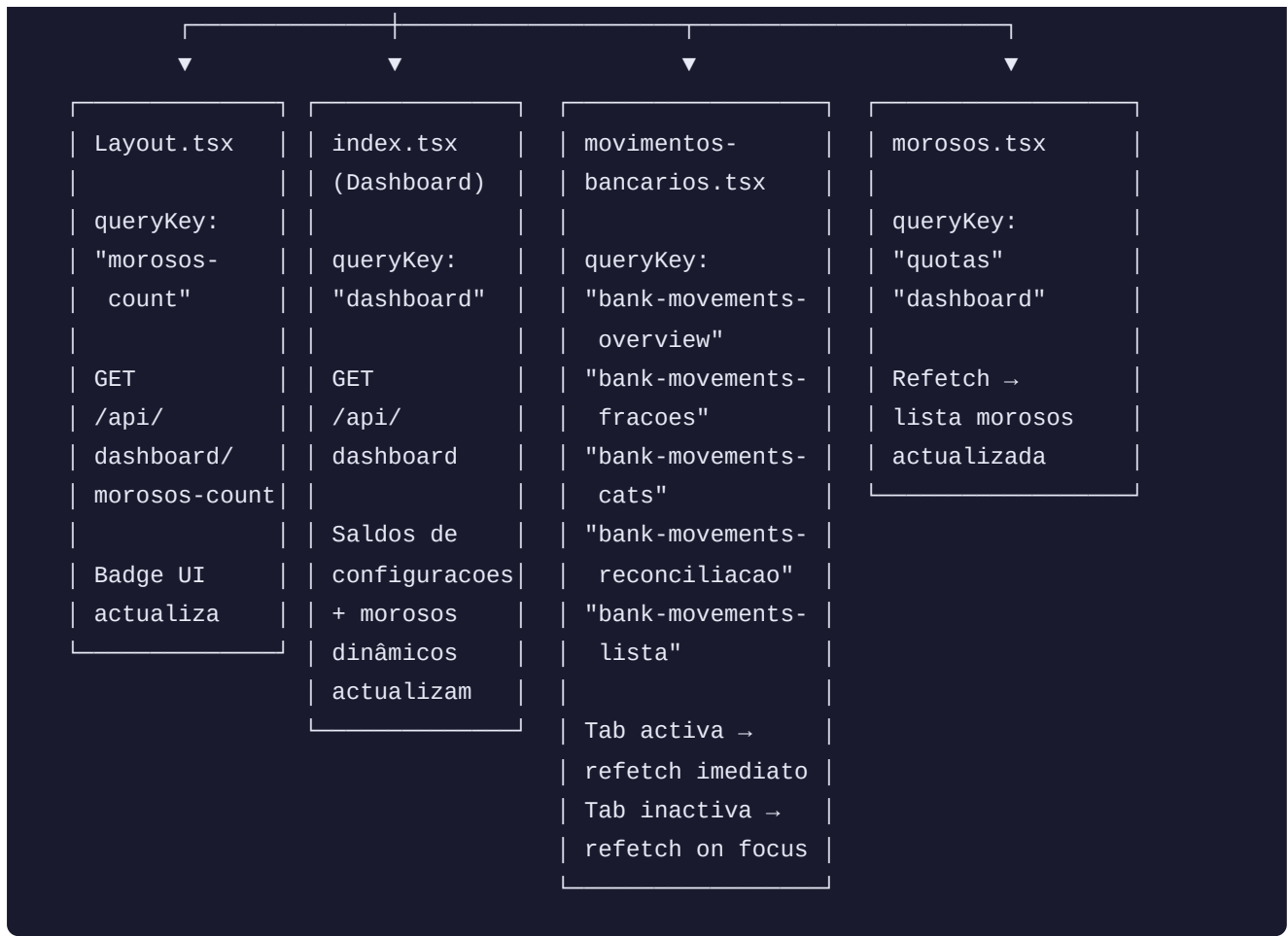
await queryClient.invalidateQueries()

→ marca TODOS os queries como stale (staleTime ignorado)

→ queries com enabled:true e observadores activos → refetch

→ queries com enabled:false → ficam stale, refetch quando

enabled mudar para true (ex: tab Reconciliação inactiva)



Nota sobre `aReceber` no Dashboard (fix 7471e3d)

ANTES (código morto):

```
portaoMorososDinamico.length >= 0 ← always true (Array.length ≥ 0)
→ aReceber = portaoAReceberDinamico ✓ (valor certo, mas por acidente)
→ fallback saldos.a_receber_portao ✗ (nunca executava)
```

DEPOIS:

```
aReceber = portaoAReceberDinamico ← directo, sem ternário morto
```

Tech Debt Catalog

Padrão identificado: **non-null assertion (!)** após `.has()` check

Logicamente seguro em todos os casos (o `!` só é atingido após `if (!map.has(id)) map.set(id, ...)` garantir que a chave existe), mas antipadrão TypeScript que esconde a intenção e é frágil a refactorizações futuras.

Ocorrências

#	Ficheiro	Linha	Mapa	Padrão
1	dashboard.ts	368	morososMap	morososMap.get(id)!.quotas.push(...)
2	dashboard.ts	369	morososMap	morososMap.get(id)!.total += ...
3	dashboard.ts	393	morososObras	morososObras.get(id)!.quotas.push(...)
4	dashboard.ts	394	morososObras	morososObras.get(id)!.total += ...
5	dashboard.ts	429	morososExtra	morososExtra.get(id)!.quotas.push(...)
6	dashboard.ts	430	morososExtra	morososExtra.get(id)!.total += ...
7	relatorio.ts	186	morososMap	morososMap.get(id)!.meses.push(...)
8	relatorio.ts	187	morososMap	morososMap.get(id)!.total += ...

Total: 8 ocorrências em 2 ficheiros.

Refatorização recomendada

O padrão `Map.has() + Map.get()!` faz duas lookups onde uma chega. Substituir por `Map.get()` com optional chaining e guard:

```
// ANTES (antipadrão)
if (!morososMap.has(id)) {
  morososMap.set(id, { fracao: row.fracao, quotas: [], total: 0 });
}
morososMap.get(id)!.quotas.push(row.quota);
morososMap.get(id)!.total += row.quota.valor;

// DEPOIS (idiomático – 1 lookup)
if (!morososMap.has(id)) {
  morososMap.set(id, { fracao: row.fracao, quotas: [], total: 0 });
}
const entry = morososMap.get(id);
if (!entry) continue; // impossível, mas satisfaz o type checker
entry.quotas.push(row.quota);
entry.total += row.quota.valor;
```

Prioridade: Baixa. Sem risco de crash. Agendar para próximo sprint de qualidade.

Verificação de Sanidade — Veredicto Final

1. Resiliência a DB vazia ☐

`getSaldos()` em `dashboard.ts` tem `try/catch` que retorna `SALDO_DEFAULTS` completos se a DB falhar ou estiver vazia. Nenhuma rota do dashboard pode devolver `undefined` ou crashar por ausência de dados.

```
DB vazia → getSaldos() retorna SALDO_DEFAULTS hardcoded
DB com dados parciais → SALDO_DEFAULTS merged com rows da DB
DB unreachable (Turso timeout) → catch → SALDO_DEFAULTS
```

O mesmo padrão existe em `relatorio.ts` (função `getSaldos` local).

2. Anti-loop no Sync ☐

Dupla guarda independente em `ProtectedRoute.tsx` :

- `sessionStorage("bank-sync-done")` — persiste entre re-renders e navegação (por sessão browser)
- `syncedRef.current` — guard por mount de componente (reset só se componente desmontar)

O `invalidateQueries()` global não dispara novo sync porque as guards bloqueiam antes do `fetch` .

3. Scope das variáveis em `movimentos-bancarios.tsx` ☐

`tab` (estado da tab activa) e `reconcData` (dados de reconciliação) estão correctamente scoped dentro de `MovimentosBancariosPage()` . O `KpiCard` não tem acesso nem dependência destas variáveis. Sem interferências entre componentes.

4. Cascata Invalidate → Refetch ☐

`invalidateQueries()` sem argumentos (React Query v5) força `stale=true` imediato em todos os queries registados, ignorando `staleTime: 60_000` . Queries com observadores activos (componente visível) fazem refetch imediato. Queries com `enabled: false` ou sem observadores ficam stale e refazem o fetch quando activados — comportamento correcto e esperado.

Sumário de Commits desta Auditoria

Commit	Fix
2bf5800	Migrar <code>loadMovimentos</code> para React Query — UI responde a <code>invalidateQueries()</code>
77258a7	<code>recalcularSaldos</code> chamado após bank sync — dashboard actualiza na DB
b509c91	Auditoria <code>recalcularSaldos</code> + UI badge morosos
0df40a2	4 correcções residuais pós-QA
7471e3d	Remover ternário <code>always-true</code> <code>portaoAReceber</code> ← commit de encerramento

Estado Final

Eixo	Estado
Lógica de dados (Dashboard)	☐ Correcto
Sync bancário (Enable Banking)	☐ Correcto
Anti-loop (<code>silentBankSync</code>)	☐ Correcto
Tab Reconciliação (scope UI)	☐ Correcto
Fallbacks DB vazia	☐ Correcto
TypeScript (<code>tsc --noEmit</code>)	☐ 0 erros
Tech debt residual	⚠ 8 non-null assertions — baixo risco, backlog

Classificação: PRODUÇÃO ☐